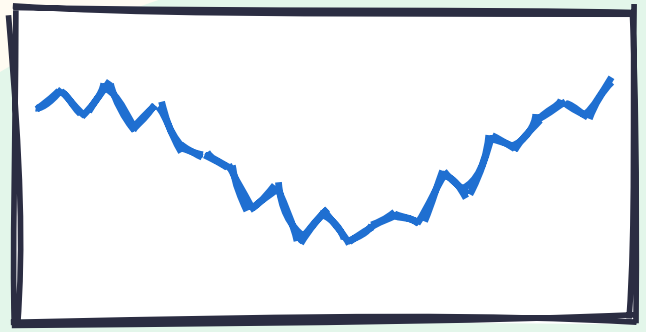
Tech Leaders · Class 9

Sensor Data Logging & Analysis Pipeline

From raw readings to evidence you can trust

ESP32 sensors | Python logging | pandas cleaning | Charts & stats





Did you know?
Data scientists often say about 80% of their time goes on collecting and cleaning data, not on the 'clever' analysis.

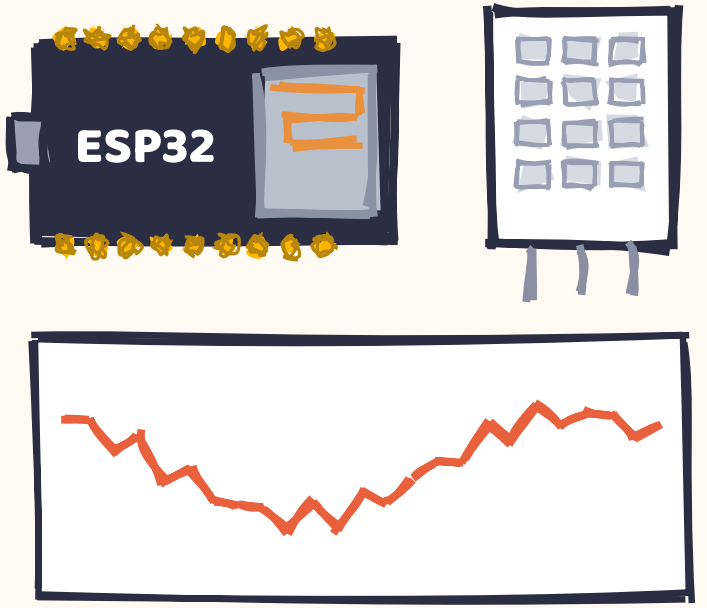
LEARN

Let's think...

Is our classroom too hot to learn in?

Opinions vs evidence

Some students say the lab gets stuffy and loud after lunch. The teacher says it is fine. Everyone has an opinion. How could we **prove** it either way, with data nobody can argue with?



Take a vote?

Check once?

Log data all day!

Today: you build a full **data pipeline**: ESP32 sensors → Python logger → CSV → pandas analysis → a chart and a conclusion.

Learning objectives

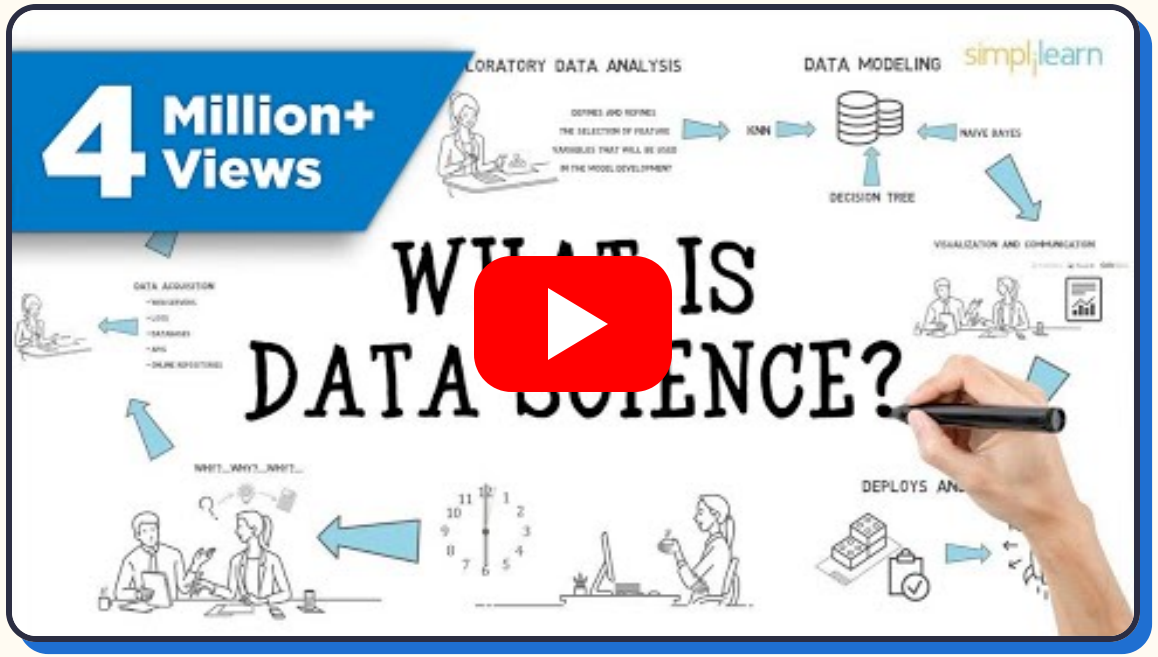
By the end of this class I can...

1	**Pipeline:** describe the 5 stages of a **data pipeline**: collect, transmit, store, clean, analyse	**Proof:** label each stage on our system diagram
2	**Collect & log:** stream timestamped sensor readings from an ESP32 into a **CSV file** with Python	**Proof:** a CSV with at least 300 rows from our room
3	**Clean:** find and remove **missing values** and **outliers** with pandas	**Proof:** we can say how many rows we dropped and why
4	**Analyse:** calculate summary stats, a **rolling average** and **correlations**, and plot them	**Proof:** a 3-panel chart saved as report.png
5	**Conclude:** make a claim backed by evidence and state its **limitations**	**Proof:** a 3-sentence finding with one limitation

LEARN

Objective 1 · Pipeline · video

Video time!



Data Science In 5 Minutes

Simplilearn · about 5 minutes · youtu.be/X3paOmcrTjQ

Watch for...

1. What are the **stages** of the data science life cycle?
2. Why is **data cleaning** such a big part of the job?
3. Give one example of a decision made from data.

Scan to watch

After the video: With your partner, write one question about our school that data could answer, and what sensor or survey you would need.

LEARN

Objective 1 · Pipeline

A pipeline: data flows through stages

The idea

Collect (sensors) → **Transmit** (USB serial) → **Store** (CSV file) → **Clean** (remove bad rows) → **Analyse** (stats and charts) → **Communicate** (a conclusion).

Each stage can break the next. A loose wire in stage 1 becomes a wrong conclusion in stage 6. Good engineers check data at **every** stage.

GIGO

Garbage in, garbage out: no analysis can rescue bad data.

Example: a water treatment plant



River water is pumped in, filtered, cleaned, tested and piped to homes. Skip the filter and dirty water reaches your tap, even if every pipe after it is perfect. Data pipelines work the same way.

LEARN

Real world

Pipelines that run the world



Weather services
Thousands of stations log every minute for forecasts



Hospitals
Monitors log heart rate and oxygen to spot danger early



Smart farms
Soil sensors decide when and how much to water



Air quality
Lahore's AQI maps come from logged PM2.5 sensors



Pick one example. What would happen if one sensor sent wrong numbers for a whole day?

Validation
Real pipelines automatically flag readings that are impossible (like 300 °C indoors).

LEARN

Objective 3 · Clean

Clean before you trust

The idea

Typical problems in our CSV:

- **Missing**: DHT22 sometimes returns **nan** (it was read too fast or a wire wobbled).
- **Impossible**: 85 °C indoors = a glitch.
- **Spikes**: someone breathed on the sensor or switched on a torch.

Rule of thumb: **remove** impossible values, **keep** real but unusual events, and **record** what you removed.

Honesty

Deleting data you don't like is cheating. Only remove data you can JUSTIFY as an error.

Example: marking a test

time	temp	hum	light
10:00:02	24.1	51.0	2210
10:00:04	24.1	51.2	2198
10:00:06	nan	nan	2205
10:00:08	24.2	51.1	2190

If one answer sheet is blank, you don't count it as 0%. If a mark says 150/100, it is clearly a mistake. But a student who really scored 98% is not a 'mistake' just because it is unusual. Same with data!

LEARN

Objective 4 · Analyse

Reading the evidence

The idea

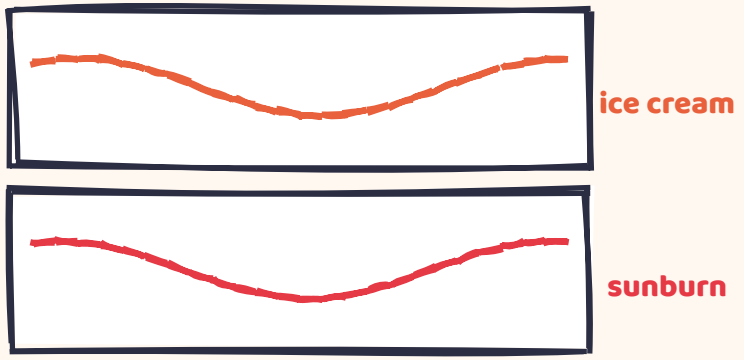
describe() gives mean, min, max and spread. Compare **morning vs afternoon**: is the difference bigger than the sensor's accuracy (DHT22 ≈ ±0.5 °C)?

Correlation is not causation. Light and temperature may rise together because the **sun** causes both, not because light heats the room.

Accuracy

A 0.3 °C difference with a ±0.5 °C sensor is NOT evidence of anything.

Example: ice cream and sunburn



Ice cream sales and sunburn cases rise together. Ice cream doesn't cause sunburn: hot sunny days cause both. Always ask: is there a hidden third factor?

Part 2 · Make it

Now let's put it to work!

You know the ideas. Time to play, build and test. Use these slides while you work:

1. Spot the bad data
2. Our logger kit (per group)
3. Wire three sensors
4. ESP32: print one CSV line every 2 s
5. Python: log to a CSV file
6. pandas: clean, summarise, plot
7. Write a finding like a scientist

MAKE

Class game

Spot the bad data

How to play

1. Teacher shows a printed CSV of 'yesterday's lab readings' with hidden problems.
2. In teams, find: a **missing** value, an **impossible** value, a **duplicate** row, a **wrong unit**, and a **time gap**.
3. Circle each one and write a rule a computer could use to catch it.
4. Fastest team with all 5 correct wins.

Then discuss

Which problems could a computer catch automatically? Which need a human to notice?

time	temp	hum	light
10:00:02	24.1	51.0	2210
10:00:04	24.1	51.2	2198
10:00:06	nan	nan	2205
10:00:08	24.2	51.1	2190

Big idea

Every cleaning rule you write here becomes a line of pandas code later.

Build it

Our logger kit (per group)

ESP32 + USB
Reads all 3 sensors

DHT22
Temperature and humidity

LDR module
Light level

KY-037 sound
Noise level

Laptop + Python
pyserial, pandas, matplotlib

Breadboard + wires
Connections

Install once: **pip install pyserial pandas matplotlib**. Arduino IDE needs the ESP32 board package and the DHT sensor library.

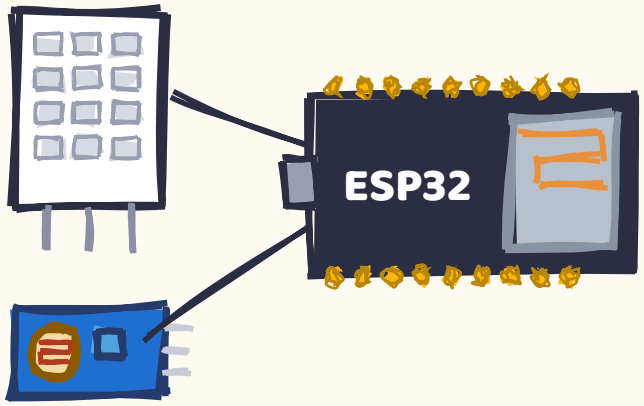
Leave it running
The best datasets are long. Start logging at the start of the lesson and analyse at the end.

MAKE

Objective 2 · Collect & log · Build step 1 of 4

Wire three sensors

Part	Connect to
DHT22 VCC / GND	3V3 / GND
DHT22 DATA	GPIO 4 (+10k pull-up if bare sensor)
LDR module AO	GPIO 34
KY-037 AO	GPIO 35
LDR & KY-037 VCC / GND	3V3 / GND



Why GPIO 34 and 35?

They are **input-only ADC1** pins. ADC2 pins stop working when Wi-Fi is on, so it is a good habit to avoid them.

Objective 2 · Collect & log · Build step 2 of 4

ESP32: print one CSV line every 2 s

```
#include "DHT.h"
DHT dht(4, DHT22);

void setup() {
  Serial.begin(115200);
  dht.begin();
  Serial.println("ms,temp_c,humidity,light,sound");
}

void loop() {
  float t = dht.readTemperature();
  float h = dht.readHumidity();
  int light = analogRead(34);          // 0..4095

  int lo = 4095, hi = 0;               // sound: peak-to-peak
  unsigned long start = millis();
  while (millis() - start < 50) {
    int v = analogRead(35);
    lo = min(lo, v);  hi = max(hi, v);
  }
  Serial.printf("%lu,%.1f,%.1f,%d,%d\n", millis(), t, h, light, hi - lo);
  delay(2000);                         // DHT22 needs >= 2 s
}
```

CSV format

Values separated by commas, same order every time, with a **header** line first.

Why peak-to-peak?

Sound is a wave. One reading could catch it anywhere. The **difference** between highest and lowest in 50 ms shows loudness.

Check

Serial Monitor at 115200 shows lines like **40123,24.1,51.0,2210,37**.

MAKE

Objective 2 · Collect & log · Build step 3 of 4

Python: log to a CSV file

```
import serial, csv, os
from datetime import datetime

PORT, FILE = "COM5", "lab_log.csv"     # Mac: /dev/cu.usbserial-...
ser = serial.Serial(PORT, 115200, timeout=3)
new_file = not os.path.exists(FILE)

with open(FILE, "a", newline="") as f:
    w = csv.writer(f)
    if new_file:
        w.writerow(["time", "temp_c", "humidity", "light", "sound"])
    while True:
        line = ser.readline().decode(errors="ignore").strip()
        parts = line.split(",")
        if len(parts) != 5 or not parts[0].isdigit():
            continue                       # skip header / broken lines
        stamp = datetime.now().isoformat(timespec="seconds")
        w.writerow([stamp] + parts[1:])
        f.flush()                          # save even if we crash
        print(stamp, parts[1:])
```

Real timestamps

The ESP32 only knows milliseconds since power-on. The laptop adds the **real time**.

Close Serial Monitor!

Only one program can use the port. If Python says 'access denied', close the Arduino Serial Monitor.

Stop

Press **Ctrl+C** to stop. The file is safe because of f.flush().

MAKE

Objective 4 · Analyse · Build step 4 of 4

pandas: clean, summarise, plot

```
import pandas as pd
import matplotlib.pyplot as plt

df = pd.read_csv("lab_log.csv", parse_dates=["time"])
before = len(df)
df = df.dropna()                                    # missing values
df = df[df.temp_c.between(5, 45) & df.humidity.between(5, 100)]
print("Removed", before - len(df), "bad rows of", before)

print(df.describe().round(1))                       # mean, min, max...
df["temp_smooth"] = df.temp_c.rolling(15).mean()    # 30-second average
print(df[["temp_c", "humidity", "light", "sound"]].corr().round(2))

fig, ax = plt.subplots(3, 1, sharex=True, figsize=(9, 7))
ax[0].plot(df.time, df.temp_c, alpha=.3); ax[0].plot(df.time, df.temp_smooth)
ax[0].set_ylabel("Temp (°C)")
ax[1].plot(df.time, df.light, color="orange"); ax[1].set_ylabel("Light")
ax[2].plot(df.time, df.sound, color="purple"); ax[2].set_ylabel("Sound")
fig.suptitle("Robotics lab, " + str(df.time.dt.date.iloc[0]))
plt.savefig("report.png", dpi=150)
```

Report the cleaning

Always say how many rows you removed and why. Readers need to know.

Rolling mean

Averages each point with its neighbours. It hides sensor noise so the **trend** is visible.

corr()

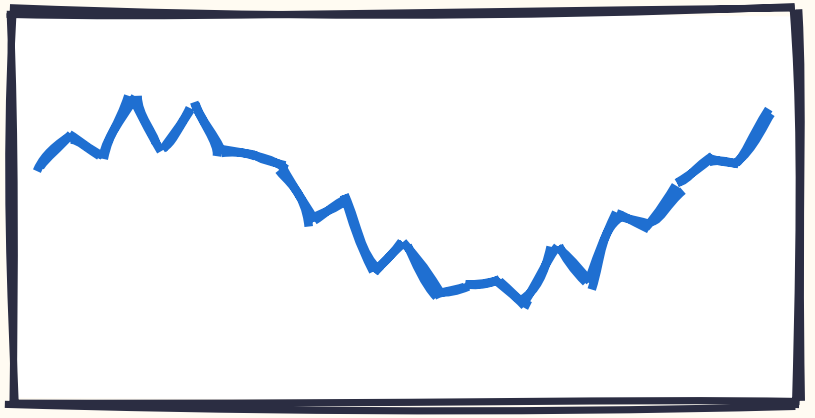
+1 = rise together, −1 = one rises as the other falls, 0 = no link.

Objective 5 · Conclude

Write a finding like a scientist

Claim → Evidence → Limitation

Claim: The lab is warmer after lunch.
Evidence: Mean temperature rose from 24.1 °C (9–11 am) to 27.3 °C (1–3 pm), across 1,840 cleaned readings. Sound peaks matched class changeovers.
Limitation: One day only, one sensor near the window. We should log for a week and move the sensor to the centre.



Be specific
Not 'it got hotter'. Say how much, when, with how many readings, and what could be wrong.

Test like an engineer

Pipeline check

Stage	✓ / ✗	Notes
ESP32 prints clean CSV lines		
Python file grows every 2 s		
300+ rows collected		
Rows removed and reason		
report.png has 3 panels		
Finding with a limitation		

Debug checklist

All temperatures nan? DHT data pin wrong or library set to DHT11. Light always 4095? Sensor on an ADC2 pin or VCC missing.

Test each stage alone first. Is the problem in the sensor, the serial line, the file, or the code?

MAKE

Level up!

Data challenges

Choose your challenge



Wireless: send readings to a Google Sheet over Wi-Fi instead of USB.



Week long: log for 5 days and compare each day's 1 pm temperature.



Two rooms: two teams log different rooms at the same time. Which is better for learning?



Auto alerts: print a warning when the 5-minute average goes above 28 °C.

Think like an engineer

Our DHT22 is ±0.5 °C. How many readings would you need to trust a 0.2 °C difference in averages?

Boss level

Build a live chart that updates while logging, using matplotlib animation.

Think about it

Data about people needs care

Our sound sensor measures noise, not words.

But imagine a school installs microphones that record **what** students say, to 'measure classroom behaviour'. Is that OK? Who should decide? Who can see the data? How long is it kept?

Good data practice

- ✔ Collect only what you **need**
- ✔ Tell people what is collected and why
- ✔ Keep it secure, delete it when done
- ✔ Report honestly, including limitations

Debate (5 min)

Should a school use sensors to automatically rank classrooms by noise? What could go wrong?

Objective check

Can I really do it? Prove it.

01	Name the 5 pipeline stages and point to each one in your setup.	○ ○ ○
02	Open your CSV. How many rows, and how often was a reading taken?	○ ○ ○
03	How many rows did you remove, and what rule did you use?	○ ○ ○
04	Show report.png. What does the rolling average show that raw data hides?	○ ○ ○
05	Read your finding: claim, evidence and one limitation.	○ ○ ○

Rate yourself

Colour one circle each. Red: not yet. Amber: with help. Green: I can explain it to a friend.

Teacher: Mini-conference: each team gets 90 seconds to present its finding and chart.

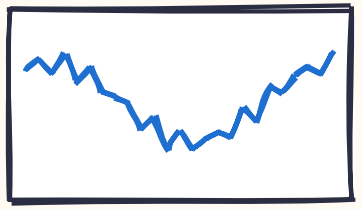
Show what you know

What did we learn?

- [x] A pipeline: collect, transmit, store, clean, analyse, communicate
- [x] CSV with a header and real timestamps makes data reusable
- [x] Clean honestly: remove errors, keep real events, report what you removed
- [x] Rolling averages show trends; correlation is not causation
- [x] A good finding has a claim, evidence and a limitation

At home

Find one news story that uses data. Does it give a limitation? Bring it next class.



Exit ticket: your temperature column has 40 'nan' values. What causes them, and what do you do?

Next class:
Machine Learning Rain Predictor!